COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 8, 2026
NAME:	Gresola, Aryanna Keith G	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Sorting Algorithm Part 1**

1. PROCEDURE OUTPUT

Preparation Task

INPUT

sorting.h:

```
sorting.h > ...
1   #ifndef SORTING_H
2   #define SORTING_H
3   #include <iostream>
4
5   // display func:
6   template <typename T>
7   void display(T arr[], int size){
8       for(int i = 0; i < size; i++){
9           std::cout << arr[i] << " ";
10          if((i + 1) % 10 == 0){
11              std::cout << std::endl;
12          }
13      }
14  }
15
16  #endif
```

main.cpp:

main.cpp > main()

```cpp
#include <iostream>
#include <stdlib.h>
#include <time.h>
#include "sorting.h"
#include "sortingAlgo.h"

const int SIZE = 100;

int main(){
    int numbers[SIZE];
    srand(time(0));
    // generating 100 random numbers:
    for(int i = 0; i < SIZE; i++){
        numbers[i] = rand() % 100 + 1;
    }

    // display:
    std::cout << "Array Elements (random numbers): " << std::endl;
    display(numbers, SIZE);
    std::cout << std::endl;
```

OUTPUT

```
Array Elements (random numbers):
37 7 40 47 69 66 45 28 5 61
68 89 44 18 27 89 33 14 34 43
12 87 3 66 85 6 97 54 89 19
23 78 62 41 13 50 2 86 19 95
95 19 25 56 97 8 93 100 85 70
66 17 45 20 41 31 19 43 87 37
3 26 99 31 39 6 36 100 17 31
88 86 83 86 22 1 42 39 21 100
41 30 46 18 63 94 8 19 56 34
94 1 48 88 49 70 8 38 94 83
```

OBSERVATION AND ANALYSIS

The program first generates 100 random numbers and stores them in the array. Since SIZE is set to 100, the loop in main.cpp runs 100 times. After generating the numbers, the display() function also goes through all 100 elements to show them on the screen.

At this part, there are still no sorting passes or comparisons because the program is only generating and displaying the numbers. The loops simply run n times, so when n = 100, there are 100 iterations. The time complexity for both generating and displaying the array is O(n) because the number of loops depends on the size of the array.

A.1. Bubble Sort Technique

INPUT

sortingAlgo.h

```
5   template <typename T>
6   void bubbleSort(T arr[], int maxSize){
7       for (int i = 0; i < maxSize; i++){
8           for (int j = i; j < maxSize; j++){
9               if(arr[i] > arr[j]){
10                  std::swap(arr[i], arr[j]);
11
12              }
13          }
14      }
15  }
16
```

main.cpp

```
22      // bubble sort:
23      std::cout << "Bubble Sort: " << std::endl;
24      bubbleSort(numbers, SIZE);
25      display(numbers, SIZE);
26      std::cout << std::endl;
```

OUTPUT

```
Bubble Sort:
1 1 2 3 3 5 6 6 7 8
8 8 12 13 14 17 17 18 18 19
19 19 19 19 20 21 22 23 25 26
27 28 30 31 31 31 33 34 34 36
37 37 38 39 39 40 41 41 41 42
43 43 44 45 45 46 47 48 49 50
54 56 56 61 62 63 66 66 66 68
69 70 70 78 83 83 85 85 86 86
86 87 87 88 88 89 89 89 93 94
94 94 95 95 97 97 99 100 100 100
```

OBSERVATION AND ANALYSIS

For the bubble sort, two nested loops are used to compare the elements using arr[i] > arr[j]. Since there are 100 elements, the comparisons start with 100, followed by 99, 98, and continue until 1. By adding 100 + 99 + 98 + until 1, I got a total of 5,050 comparisons. This can also be computed using the formula n(n+1) / 2. When n = 100, the computation is 100(101) / 2 = 5,050 comparisons. Whenever arr[i] > arr[j] is true, the program swaps the two elements. This process continues until the smaller numbers are moved toward the beginning and the array becomes sorted in ascending order.

For the time complexity, bubbleSort() is O(n^2) because of the nested loops. In main.cpp, calling the bubble sort is also O(n^2), while displaying the array is O(n). For this specific code, both the best and worst cases are O(n^2) because the loops still run completely even if the array is already sorted.

A.2. Selection Sort Algorithm

INPUT

sortingAlgo.h

```
18  template <typename T>
19  int routine_smallest(T arr[], int k, int arrSize){
20      int position = k;
21      T smallestElement = arr[k];
22
23      for (int j = k + 1; j < arrSize; j++){
24          if(arr[j] < smallestElement){
25              smallestElement = arr[j];
26              position = j;
27          }
28      }
29      return position;
30  }
31
32
33  template <typename T>
34  void selectionSort(T arr[], int arrSize){
35      int min;
36      for (int i = 0; i <arrSize; i++){
37          min = routine_smallest(arr, i, arrSize);
38          std::swap(arr[i], arr[min]);
39      }
40  }
```

main.cpp

```
28      // selection sort:
29      std::cout << "Selection Sort: " << std::endl;
30      selectionSort(numbers, SIZE);
31      display(numbers, SIZE);
32      std::cout << std::endl;
```

OUTPUT

```
Selection Sort:
1 1 2 3 3 5 6 6 7 8
8 8 12 13 14 17 17 18 18 19
19 19 19 19 20 21 22 23 25 26
27 28 30 31 31 31 33 34 34 36
37 37 38 39 39 40 41 41 41 42
43 43 44 45 45 46 47 48 49 50
54 56 56 61 62 63 66 66 66 68
69 70 70 78 83 83 85 85 86 86
86 87 87 88 88 89 89 89 93 94
94 94 95 95 97 97 99 100 100 100
```

OBSERVATION AND ANALYSIS

In the selection sort, the algorithm goes through the array 100 times because the outer loop checks each element. In every pass, the routine_smallest() function checks the remaining elements to find the smallest value. The comparisons start with 99, then 98, 97, and continue until 1, giving a total of 4,950 comparisons. This can also be computed using the formula n(n-1) / 2. When n = 100, the computation is 100(99) / 2 = 4,950 comparisons.

The algorithm first assumes that the current element is the smallest and then compares it with the remaining elements using arr[j] < smallestElement. If a smaller value is found, its position is saved and it is swapped with the current element. By repeating this process, the smallest values are placed at the beginning until the whole array is sorted. The time complexity is O(n^2) for both the best and worst cases because it still checks the remaining elements even when the array is already sorted.

A.3. Insertion Sort Algorithm

INPUT

sortingAlgo.h:

```
42  template <typename T>
43  void insertionSort(T arr[], int arrSize){
44      int k = 1, j; // assuming the index 0 is already sorted.
45      T temp;
46
47      while(k < arrSize){
48          temp = arr[k];
49          j = k - 1;
50          while(j >= 0 && temp <= arr[j]){
51              arr[j + 1] = arr[j];
52              j--;
53          }
54          arr[j + 1] = temp;
55          k++;
56      }
57  }
58
59  #endif
```

main.cpp

```
34      // insertion sort:
35      std::cout << "Insertion Sort: " << std::endl;
36      insertionSort(numbers, SIZE);
37      display(numbers, SIZE);
38      std::cout << std::endl;
```

OUTPUT

```
Insertion Sort:
1 1 2 3 3 5 6 6 7 8
8 8 12 13 14 17 17 18 18 19
19 19 19 19 20 21 22 23 25 26
27 28 30 31 31 31 33 34 34 36
37 37 38 39 39 40 41 41 41 42
43 43 44 45 45 46 47 48 49 50
54 56 56 61 62 63 66 66 66 68
69 70 70 78 83 83 85 85 86 86
86 87 87 88 88 89 89 89 93 94
94 94 95 95 97 97 99 100 100 100
```

OBSERVATION AND ANALYSIS

For insertion sort, the process starts with the second element because the first element is already considered sorted. Since there are 100 elements, the outer while loop makes 99 passes, starting from k = 1 until k = 99. The number of comparisons depends on how the numbers are arranged. In the worst case, it can make up to 4,950 comparisons, which comes from 1 + 2 + 3 until 99. This can also be computed using the formula n(n-1) / 2. When n = 100, the computation is 100(99) / 2 = 4,950 comparisons. In the best case, when the array is already sorted, there is only one comparison for each pass, resulting in approximately n-1, or 99 comparisons.

The algorithm temporarily stores the current element in temp and compares it with the elements before it using temp <= arr[j]. If the previous element is greater than or equal to temp, it is moved one position to the right until the correct position for temp is found. By repeating this process, more elements are gradually placed in the correct order until the whole array is sorted. The best-case time complexity is O(n) when the array is already sorted, while the worst case is O(n^2) when the elements require the maximum number of comparisons and movements.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CHOSEN SORTING ALGORITHM AND JUSTIFICATION

Sorting Algorithm: Insertion Sort
Justification: I chose Insertion Sort because I think it is more suitable for the data used in this activity. We only have 101 votes, and the values are limited from 1 to 5, which means there are many repeated values and the data is relatively simple to sort. Insertion Sort works well with smaller sets of data because it places each element in its proper position as it goes through the array.

Compared to Bubble Sort, which repeatedly compares and swaps elements, Insertion Sort can be more practical because it only shifts elements when needed. Selection Sort, on the other hand, still needs to search through the remaining elements to find the smallest value in every pass. For this problem, I found Insertion Sort to be the best choice because the data is small and has only values from 1 to 5, making it simple and efficient enough for sorting the votes.

PSEUDOCODE

Pseudocode for MAIN.CPP

```
START

INITIALIZE ARR_SIZE = 101

INITIALIZE voteArr with ARR_SIZE elements

INITIALIZE voteCount[5 ] = {0, 0, 0, 0, 0}

INITIALIZE the names of the 5 candidates

GENERATE random votes from 1 to 5

FOR i = 0 to ARR_SIZE - 1
        STORE a random number from 1 to 5 in voteArr[i]
END FOR

DISPLAY the list of candidates

DISPLAY the unsorted votes array

SORT voteArr using INSERTION SORT

DISPLAY the sorted votes array

FOR i = 0 to ARR_SIZE - 1
        IF voteArr[i]  = 1 THEN
```

```
                INCREMENT voteCount[0]
        ELSE IF voteArr[i] == 2 THEN
                INCREMENT voteCount[1]
        ELSE IF voteArr[i] == 3 THEN
                INCREMENT voteCount[2]
        ELSE IF voteArr[i] == 4 THEN
                INCREMENT voteCount[3]
        ELSE IF voteArr[i] == 5  THEN
                INCREMENT voteCount[4]
        END IF

END FOR

DISPLAY the vote count of each candidate

INITIALIZE winner = 0

FOR i = 1 to 4
        IF voteCount[i] > voteCount[winner] THEN
                SET winner = i
        END IF
END FOR

DISPLAY the winning candidates

DISPLAY the number of votes received by the winning candidates

STOP
```

Pseudocode for SORTINGALGO.H

```
START

FUNCTION insertionSort(arr, arrSize)
        INITIALIZE variable j

        INITIALIZE temporary variable temp

        FOR i = 1 TO arrSize - 1
                STORE the current element arr[i] in temp

                SET j to the position before the current element

                WHILE j is greater than or equal to 0
                AND the element arr[j] is greater than temp

                        MOVE the element arr[j] one position to the right
                        DECREMENT j to compare the next previous element
```

END WHILE

INSERT temp into its correct position at arr[j + 1]

END FOR

END FUNCTION

STOP

INPUT

sortingAlgo.h

HOA7P1_SUPPLEMENTARY > C sortingAlgo.h > ...

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

template <typename T>
void insertionSort(T arr[], int arrSize){
    int j;
    T temp;

    for(int i = 1; i < arrSize; i++){
        temp = arr[i];
        j = i - 1;
        while(j >= 0 && arr[j] > temp){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j  + 1] = temp;
    }
}

#endif
```

main.cpp

HOA7P1_SUPPLEMENTARY > main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <string>
#include "sortingAlgo.h"

int main(){
    const int ARR_SIZE = 101;
    int voteArr[ARR_SIZE];
    int voteCount[5] = {0, 0, 0, 0, 0};

    std::string candidate[5] = {
        "Bo Dolton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    srand(time(NULL));
    // generate random votes:
    for(int i = 0; i < ARR_SIZE; i++){
        voteArr[i] = rand() % 5 + 1;
    }

    // displays candidate:
    std::cout << "--- LIST OF CANDIDATES ---" << std::endl;
    for(int i = 0; i < 5; i++ ){
        std::cout << i + 1 << ". " << candidate[i] << std::endl;
    }
    std::cout << std::endl;

    // display unsorted votes array:
    std::cout << "--- UNSORTED VOTES ARRAY ---" << std::endl;
    for(int i = 0; i < ARR_SIZE; i++){
        std::cout << voteArr[i] << " ";
        if((i + 1) % 20 == 0){
            std::cout << std::endl;
        }
    }
```

```
41      std::cout << std::endl;
42      // sorting the array using insertion sort:
43      insertionSort(voteArr, ARR_SIZE);
44      // displaying sorted votes array:
45      std::cout << "---SORTED VOTES ARRAY---" << std::endl;
46      for(int i = 0; i < ARR_SIZE; i++){
47          std::cout << voteArr[i] << " ";
48          if((i + 1) % 20 == 0){
49              std::cout << std::endl;
50          }
51      }
52
53      // count the votes:
54      for(int i = 0; i < ARR_SIZE; i++){
55          if(voteArr[i] == 1){
56              voteCount[0]++;
57          }
58          else if(voteArr[i] == 2){
59              voteCount[1]++;
60          }
61          else if(voteArr[i] == 3){
62              voteCount[2]++;
63          }
64          else if(voteArr[i] == 4){
65              voteCount[3]++;
66          }
67          else if(voteArr[i] == 5){
68              voteCount[4]++;
69          }
70
71      }
72
73      // display vote count:
74      std::cout << std::endl;
75      std::cout << "--- VOTE COUNT ---" << std::endl;
76      for(int i = 0; i < 5; i++){
77          std::cout << candidate[i] << " = " << voteCount[i] << " votes" << std::endl;
78      }
```

```
79
80      // determine the winner:
81      int winner = 0;
82      for(int i = 1; i < 5; i++){
83          if(voteCount[i] > voteCount[winner]){
84              winner = i;
85          }
86      }
87
88      // display the finalized result:
89      std::cout << std::endl;
90      std::cout << "--- FINAL RESULT---" << std::endl;
91      std::cout << "Winning Candidate: " << candidate[winner] << std::endl;
92      std::cout << "Number of Votes: " << voteCount[winner] << std::endl;
93      std::cout << "Congratulations!" << std::endl;
94
95  return 0;
96  }
```

OUTPUT

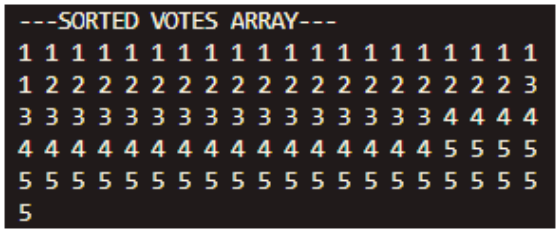
```
--- UNSORTED VOTES ARRAY ---
3 3 5 5 4 1 5 5 1 2 1 5 5 3 4 5 3 3 1 1
1 1 2 2 2 3 1 1 4 5 5 2 4 5 4 3 1 5 4 2
2 5 2 1 5 2 3 3 5 1 5 1 3 1 4 4 5 1 5 4
2 4 5 4 4 5 4 3 5 1 4 2 4 5 1 3 3 5 2 1
4 1 1 2 5 2 2 3 4 4 4 4 3 2 2 2 5 5 3 3
1
```

```
---SORTED VOTES ARRAY---
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5
```

OUTPUT

```
--- VOTE COUNT ---
Bo Dolton Capistrano = 21 votes
Cornelius Raymon Agustin = 18 votes
Deja Jayla Banaga = 17 votes
Lalla Brielle Yabut = 20 votes
Franklin Relano Castro = 25 votes

--- FINAL RESULT---
Winning Candidate: Franklin Relano Castro
Number of Votes: 25
Congratulations!
```

OUTPUT		
SORTED ARRAY	**MANUAL COUNT**	**COUNT RESULT OF ALGORITHM**
		
1	21	21
2	18	18
3	17	17
4	20	20
5	25	25

QUESTION
1. **Was your developed vote counting algorithm effective? Why or why not?** *Yes, I think my developed vote counting algorithm was effective because it made the overall vote counting process easier and more organized. By sorting the vote values first, the same numbers were grouped together, which made it easier to identify and count the votes for each candidate. Instead of manually checking the unsorted votes one by one, the algorithm automatically counted each vote and compared the results to determine the candidate with the highest number of votes. Because of this, I think the algorithm was effective for the given problem since it made the counting process more efficient, easier to verify, and less prone to errors.*

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT and Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: ChatGPT was used for code assistance and explaining concepts I did not fully understand, while Grammarly was used for grammar correction. These tools did not replace my original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.